

AgentIA

Del dolor al estado actual: propuesta, auto-mejora, diseño, orquestación, interfaz y coste

Ingeniería AgentIA

2026-09-29

El problema

Generar un microservicio de producción es **caro, lento e inconsistente**. Cada servicio debe obedecer las mismas reglas — capas estrictas, contratos inmutables, errores centralizados, compilación hermética — y cada incumplimiento se paga en revisión, defectos o deuda técnica.

El síntoma

Durante meses la plataforma reportó “**0 de 15 sesiones verificadas**”: no podía demostrar que su propia salida compilaba.

- La causa era un defecto de montaje Docker de **un solo carácter**.
- Un sistema que no verifica su propia salida no puede prometer fiabilidad.

Microservice Code Studio: un agente autónomo que convierte una especificación formal en un microservicio Spring Boot **compilado y probado**, y se niega a reportar éxito salvo que eso sea cierto.

Entrada	especificación formal (blueprint + historias BDD)
Generación	agente LangGraph de 8 etapas, gobernado por reglas
Verificación	<code>mvn test -o</code> en Docker hermético (sin red)
Reparación	acotada (máx. 3); al límite, BLOQUEADO → humano
Entrega	ZIP o publicación Git atómica

La promesa no es “código mejor”: es gobernanza, verificada y auditable.

La propuesta de auto-mejora

El bucle reflexivo (construido, features 014–015)

Un LLM reflexiona sobre los *diagnósticos* y propone ediciones a un documento de “habilidad”; se admite una edición solo si su contribución medida es positiva, y se desaloja solo por debajo de un suelo de evidencia. Sin pesos, sin GPUs.

La condición que lo gobierna

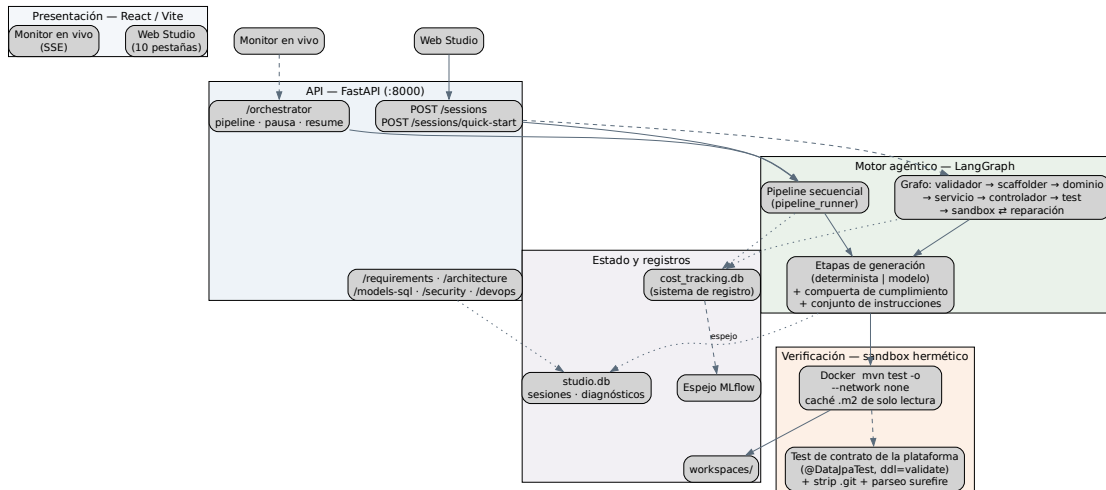
“Un bucle alimentado solo con un veredicto binario no es mejor que ningún bucle.”
(CoEvoSkills: 41.1 con oráculo opaco vs 42.4 sin evolución; 71.1 con verificador de diagnóstico.) Por eso se construyó primero el **canal de diagnóstico** y se aplazó el optimizador.

- La polaridad (prohibiciones vs prescripciones) dio **nulo**; el *contenido* de las instrucciones sí importa.
- Antes de auto-mejorar hay que hacer **determinista el verificador**.

- **Primera compilación verificada** — el defecto de montaje se corrigió; la plataforma ya verifica su salida.
- **Ambas rutas de producto verifican** — la ruta secuencial antes generaba código sin compilarlo; hoy comparte la misma veta de verificación.
- **La métrica dejó de ser manipulable** — de una densidad a una penalización absoluta relativa a una línea base congelada.
- **El verificador es intermitentemente inestable ($\approx 33\%$)**: 4/6 bajo carga, 8/8 secuencial. Causa probable: reetiquetado SELinux :Z sobre una caché compartida. *Es el siguiente punto a cerrar.*

El diseño guiado por especificación

especificación → generación → verificación contra la especificación



El diseño guiado por especificación (2)

Especificación	Qué prescribe
Blueprint / spec.md (Spec Kit)	entidades, validaciones, historias BDD, capas
Constitución (6 principios)	capas, contratos inmutables, errores, offline, gates, secretos
Documentos de instrucción (5)	el procedimiento por etapa
Lista blanca de dependencias	el límite del pom.xml
Contratos (19)	el comportamiento por feature

La especificación es la autoridad, no el modelo.

Orquestación (2): qué se antepone a cada prompt

Se antepone, en orden	Agente (pista del prompt)
prefijo de habilidad (hoy vacío)	Scaffolder: “solo dependencias de la lista blanca”
documento de instrucción (system prompt)	Dominio: “records nativos; @Data prohibido”
payload de la tarea (blueprint + previos)	Servicio: “lógica solo en servicio; inyección por constructor”
rutas que posee	Controlador: “un @RestControllerAdvice que registra la traza”
formato de respuesta (un JSON)	Test: “cada escenario se mapea a un método”

Fase	Qué ocurre
1. Requisitos	historias BDD, exportación spec.md
2. Arquitectura	componentes y contratos (Mermaid)
3. Modelos & SQL	entidades JPA, schema.sql, ER
4. Blueprints	ingesta y validación spec.md / JSON
5. Generación & Monitor	síntesis + logs en vivo (SSE)
6. Código & Fix	explorador, diffs, desbloqueo manual
7. Calidad SAST	auditoría, quality gate, remediación
8. DevOps	Dockerfile, CI/CD, k8s, despliegue local
9. Entrega	ZIP o Git atómico

- React/Vite de una sola página, **diez pestañas** en español.
- Tema **claro/oscuro**, stepper de ciclo de vida, visores **Mermaid**.
- Panel LLM: proveedor (Gemini, Groq, OpenAI, DeepSeek, mock), clave, modelo, verificación.
- Verificación honesta de tres estados; auditoría que eliminó resultados fabricados.

A demostrar	Detalle
Pestaña 5 — Monitor Live	streaming SSE en vivo
Pestañas 2–3	diagramas Mermaid (arquitectura / ER)

Alcance	Llamadas	Coste (USD)	Tok. salida
Experimento de polaridad	750	6.58	—
Cinco congelados	120	0.63	—
Diez difíciles	50	0.64	—
Sondas / sueltas	≈250	≈0.16	—
Total	1172	8.01	12.28M

- \$0.052–\$0.077 por servicio; ≈\$0.005 por llamada; todo en deepseek-flash.
- Selección guiada por verificación: **\$0.20 por servicio fiable** (k=3).

El coste (2): el espejo MLflow

Se ha desviado (y puede verse vacío)

El espejo totaliza **\$6.39** frente a **\$8.01** (subestimación del 20%). Además, mlflow ui abre el almacén vacío `mlruns/` en vez de `mlflow.db`, y los agregados de sesión (168) cayeron en el experimento legado Default, con 0 en agentia.

- Arrancar con `-backend-store-uri sqlite:///mlflow.db`.
- Reparar el targeting de experimento en `mlflow_sink.py`.

Dónde estamos

De “no puede verificar su propia salida” a “genera, compila y verifica a \$0.06 por servicio”, con 100% de cobertura con tres candidatos a \$0.20.

- ➊ Hacer **determinista el verificador** (un montaje).
- ➋ Hacer que la **medida pueda moverse** (minar fallos en reglas).
- ➌ Solo entonces, preguntar si un documento de habilidad aporta algo.

El bucle de auto-mejora queda cerrado tras esa compuerta — la misma que el diseño siempre exigió.